

Entity-Relationship Modeling

Lecture 07

Borough of Manhattan Community College

Entity–Relationship Modeling

- ❑ The *Entity–Relationship* (*ER*) is a nontechnical communication model that describes the nature of the data and how it is used by the enterprise.
- ❑ The *Entity–Relationship* modeling begins by identifying the important data called *entities* and *relationships* between the data that must be represented in the model.
- ❑ We then add more details, such as the information we want to hold about the entities and relationships called *attributes* and any *constraints* on the entities, relationships, and attributes.

Entity Types

- ❑ **Entity type:** A group of objects in the “real world” with the same properties.
- ❑ An entity type has an independent existence and can be objects with *physical existence* or objects with *conceptual existence*.
- ❑ Examples of entities with physical and conceptual existences.

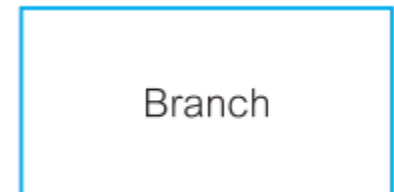
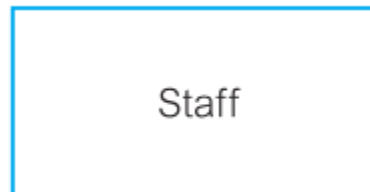
Physical existence	
Staff	Part
Property	Supplier
Customer	Product

Conceptual existence	
Viewing	Sale
Inspection	Work experience

Entity Types

- ❑ A database normally contains many different entity types. We identify each entity type by a name and a list of properties.
- ❑ Diagrammatic representation of entity types:
 - Each entity type is shown as a rectangle, labeled with the name of the entity.
 - In **UML** (Unified Modeling Language), the first letter of each word in the entity name is uppercase (for example, **Staff** and **PropertyForRent**).

Diagrammatic representation of the Staff and Branch entity types.



- ❑ **Entity occurrence:** A uniquely identifiable object of an entity type.

Relationship Types

- ❑ **Relationship type:** A set of meaningful associations among entity types.
 - A relationship type is a set of associations between one or more participating entity types.
 - Each relationship type is given a name that describes its function.



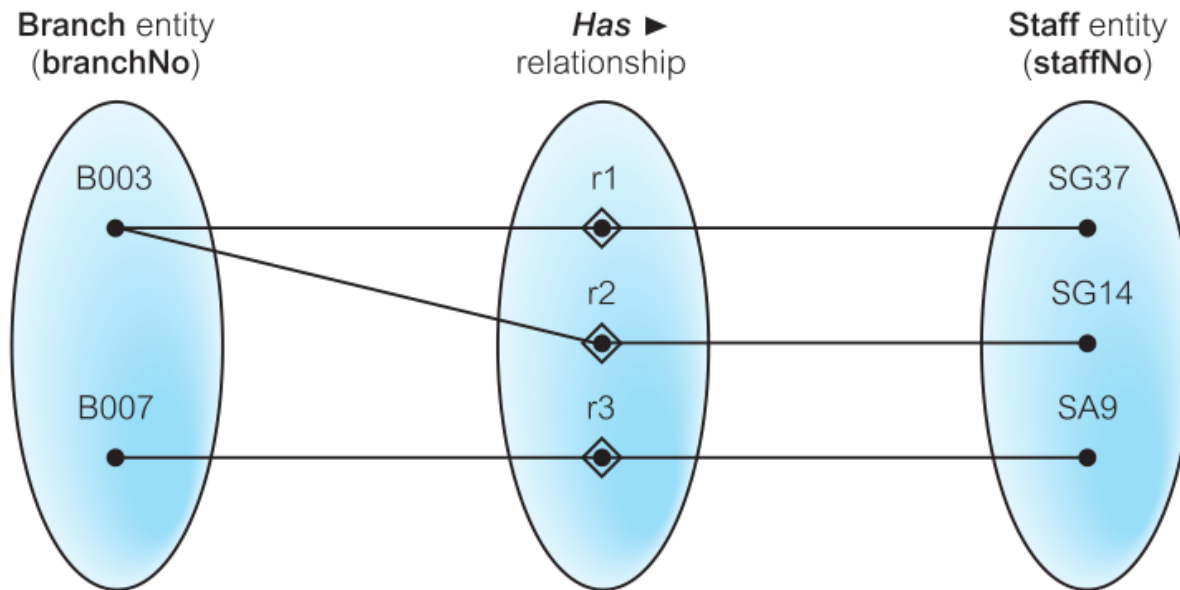
- ❑ **Relationship occurrence:** A uniquely identifiable association that includes one occurrence from each participating entity type.
- ❑ A relationship occurrence indicates the particular entity occurrences that are related.

Relationship Types

- ❑ Consider a relationship type called ***Has***, which represents an association between **Branch** and **Staff** entities, that is Branch ***Has*** Staff.
- ❑ Each occurrence of the ***Has*** relationship associates one **Branch** entity occurrence with one **Staff** entity occurrence.
- ❑ We can examine examples of individual occurrences of the ***Has*** relationship using a ***semantic net***.
- ❑ A ***semantic net*** is an object-level model, which uses the symbol • to represent entities and the symbol ♦ to represent relationships.

Relationship Types

- Relationships are represented by lines that join each participating **Branch** occurrence with the associated **Staff** occurrence.



- If we represented an enterprise using *semantic nets*, it would be difficult to understand, due to the level of details.

Relationship Types

- ❑ We can more easily represent the relationships between entities in an enterprise using the concepts of the **ER** model.
- ❑ The **ER** model uses a higher level of abstraction than the **semantic net** by combining sets of entity occurrences into entity types and sets of relationship occurrences into relationship types.



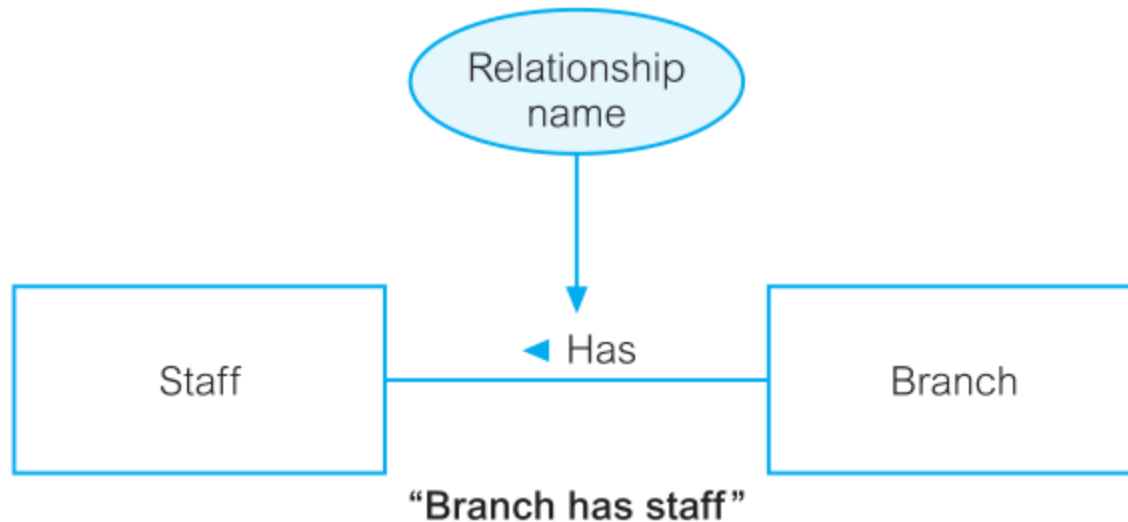
Relationship Types

□ Diagrammatic representation of relationship types:

- Each relationship type is shown as a line connecting the associated entity types and labeled with the name of the relationship.
- A relationship is named using a verb (for example, ***Supervises*** or ***Manages***) or a short phrase including a verb (for example, ***LeasedBy***).
- Again, the first letter of each word in the relationship name is shown in uppercase.
- A relationship name should be unique for a given ***ER*** model.

Relationship Types

- A relationship is only labeled in one direction, which normally means that the name of the relationship only makes sense in one direction (for example, **Branch Has Staff** makes more sense than **Staff Has Branch**).
- Once the relationship name is chosen, an arrow symbol is placed beside the name indicating the correct direction for a reader to interpret the relationship name (for example, **Branch Has ► Staff**).



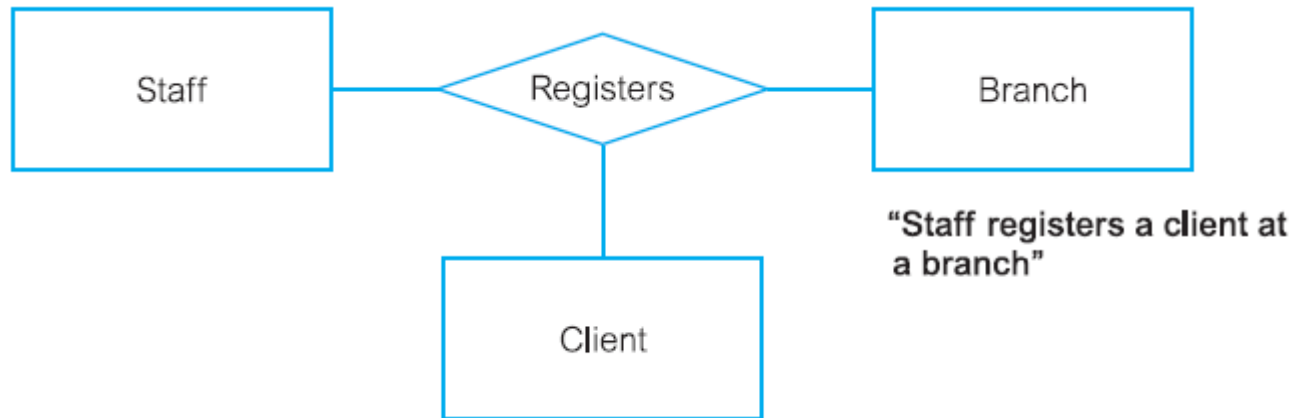
Degree of Relationship Type

- ❑ The entities involved in a particular relationship type are referred to as **participants** in that relationship.
- ❑ The number of participants in a relationship type is called the **degree** of that relationship.
- ❑ A relationship of degree two is called **binary**.
- ❑ An example of a binary relationship is the ***Has*** relationship with two participating entity types; namely, **Staff** and **Branch**.

Representation of Complex Relationships

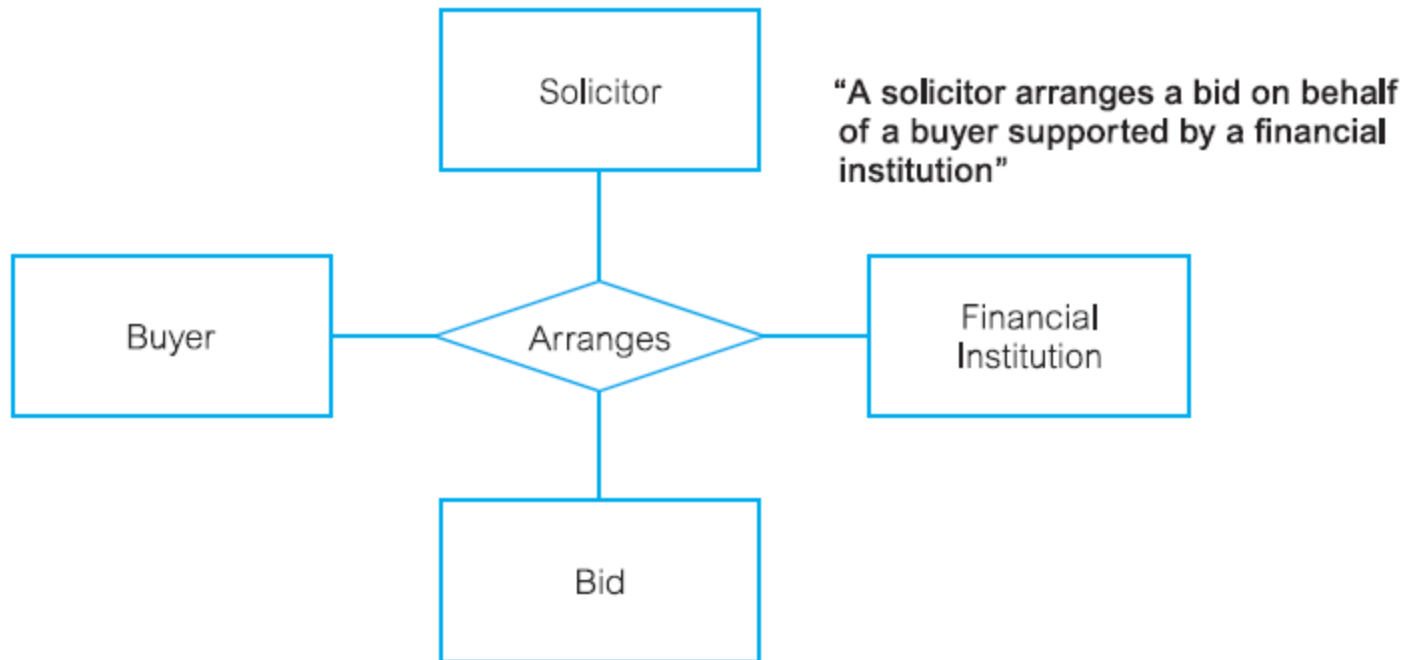
- ❑ A relationship of degree three is called **ternary**.

An example of a ternary relationship is ***Registers*** with three participating entity types: **Staff**, **Branch**, and **Client**.



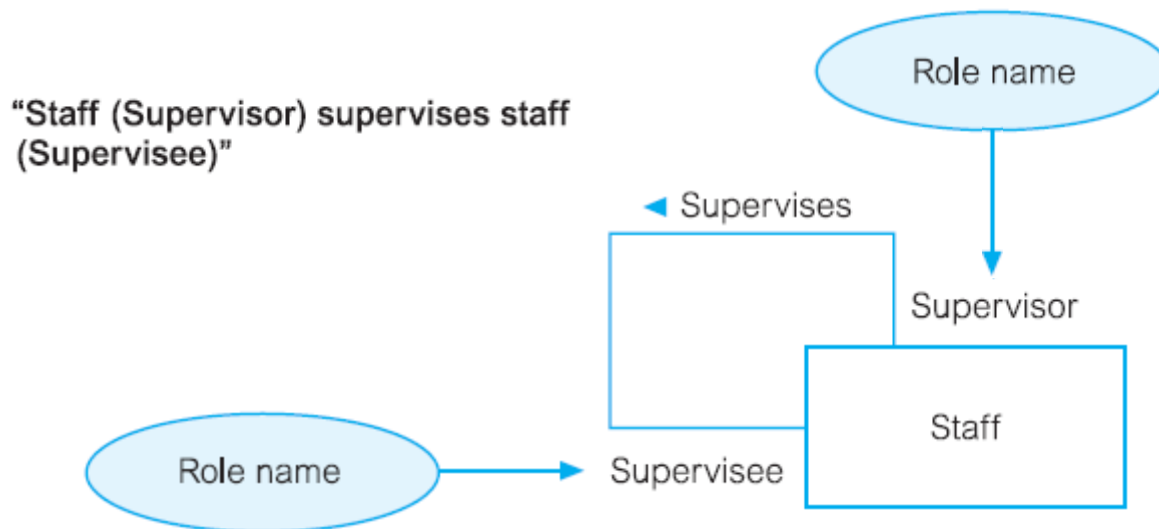
- ❑ The UML notation uses a diamond to represent complex relationships —relationships with degrees higher than binary.
- ❑ The name of the relationship is displayed inside the diamond, and in this case, the directional arrow normally associated with the name is omitted.

Representation of Complex Relationships



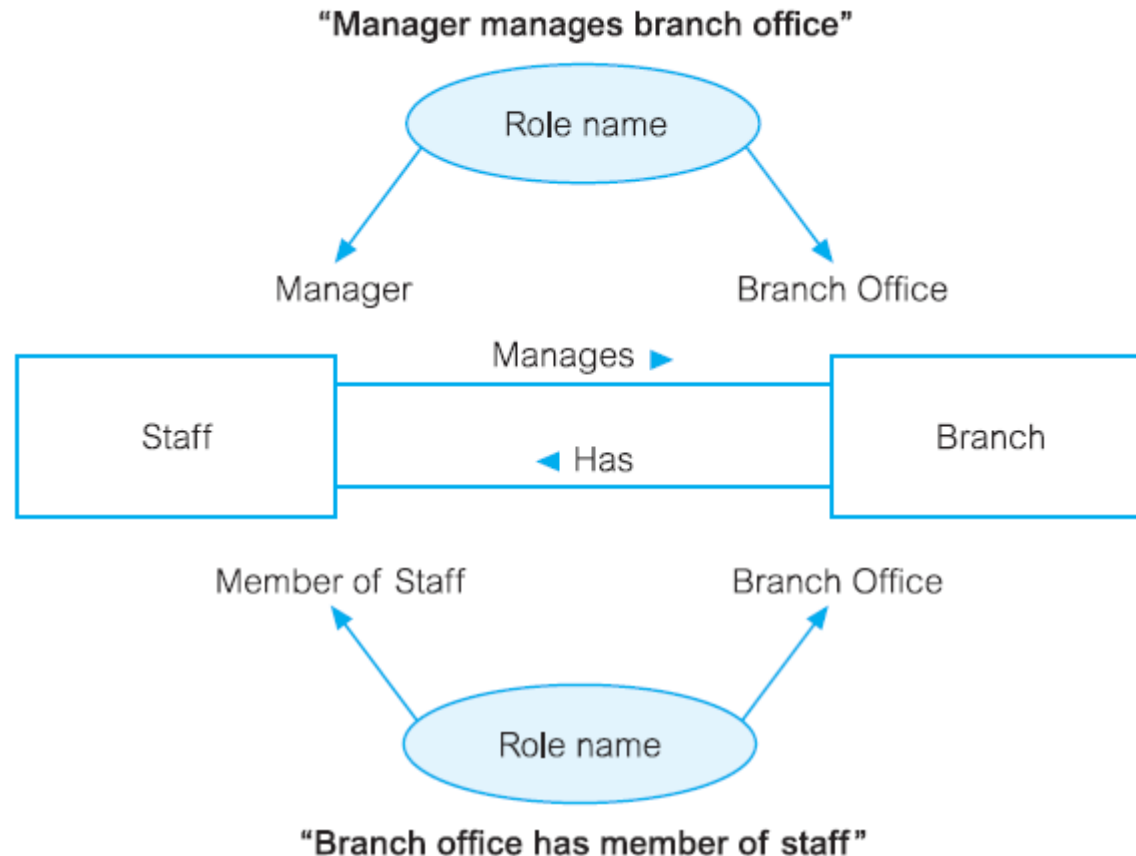
Recursive Relationship

- ❑ **Recursive relationship:** A relationship type in which the same entity type participates more than once in different roles.
- ❑ Relationships may be given role names to indicate the purpose that each participating entity type plays in a relationship.



- ❑ Recursive relationships are called **unary** relationships.

- ❑ Role names may also be used when two entities are associated through more than one relationship. For example, the **Staff** and **Branch** entity types are associated through two distinct relationships called *Manages* and *Has*.



- ❑ Role names are usually not required if the function of the participating entities in a relationship is unambiguous.

Attributes

- ❑ **Attribute:** A property of an entity or a relationship type.
- ❑ For example, a **Staff** entity type may be described by the *staffNo*, *name*, *position*, and *salary* attributes.
- ❑ The attributes hold values that describe each entity occurrence and represent the main part of the data stored in the database.
- ❑ A relationship type that associates entities can also have attributes similar to those of an entity type.
- ❑ Each attribute is associated with a set of values called a **domain**.

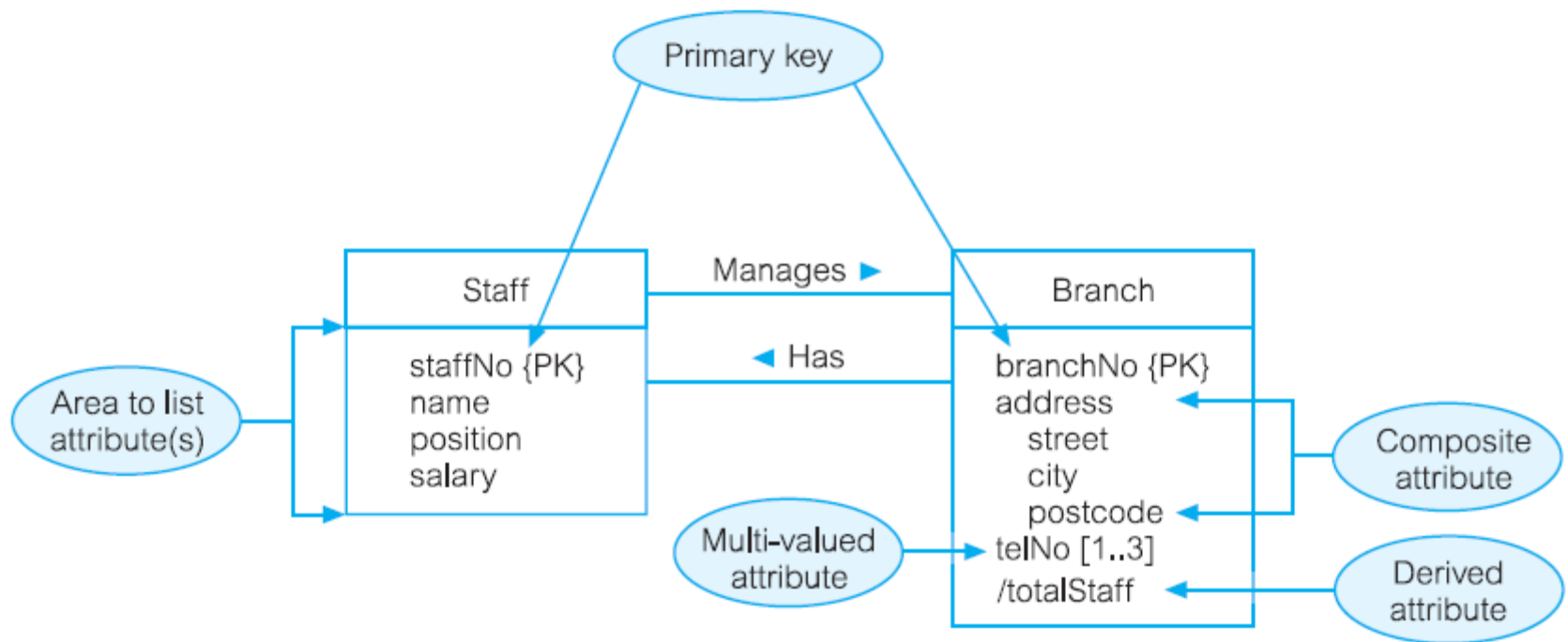
Representation of Attributes

- ❑ If an entity type is to be displayed with its attributes, we divide the rectangle representing the entity in two. The upper part of the rectangle displays the name of the entity and the lower part lists the names of the attributes.
- ❑ The first attribute(s) to be listed is the primary key for the entity type.
- ❑ The name(s) of the primary key attribute(s) can be labeled with the tag {PK}.
- ❑ In UML, the name of an attribute is displayed with the first letter in lowercase and with the first letter of each subsequent word in uppercase if the name has more than one word (for example, *address* and *telNo*).

Representation of Attributes

- ❑ For some simpler database systems, it is possible to show all the attributes for each entity type in the *ER* diagram. However, for more complex database systems, we just display the attribute, or attributes, that form the primary key of each entity type. When only the primary key attributes are shown in the *ER* diagram, we can omit the {PK} tag.
- ❑ For composite attributes, we list the name of the composite attribute followed below by the names of its simple component attributes indented to the right.
- ❑ For multivalued attributes, we label the attribute name with an indication of the range of values available for the attribute. For example, the *telNo* attribute holds one to a maximum of three values, we can label the attribute with [1..3].
- ❑ For derived attributes, we prefix the attribute name with a "/".

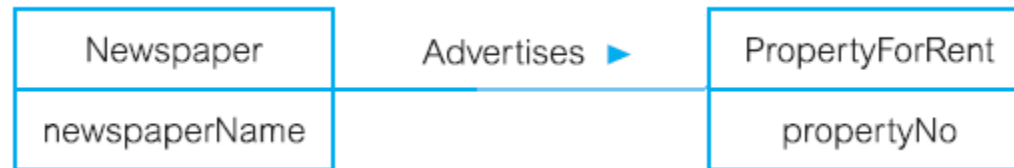
Representation of Attributes



Diagrammatic representation of Staff and Branch entities and their attributes.

Attributes on Relationships

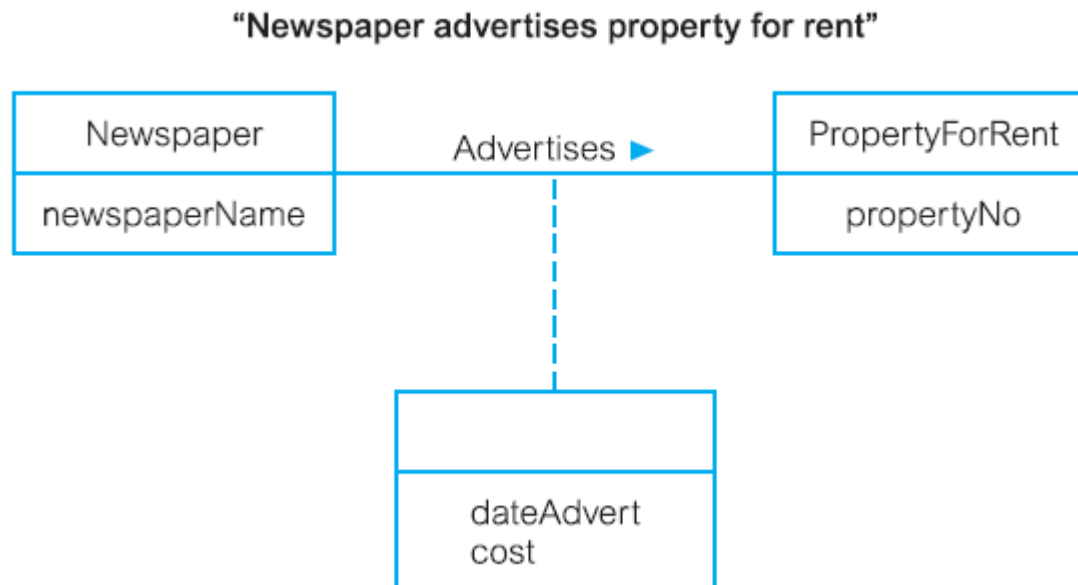
- Attributes can also be assigned to relationships. For example, consider the relationship *Advertises*, which associates the **Newspaper** and **PropertyForRent** entity types.



- To record the date the property was advertised and the cost, we associate this information with the *Advertises* relationship as attributes called *dateAdvert* and *cost*, rather than with the **Newspaper** or the **PropertyForRent** entities.

Representation of Attributes on Relationships

- ❑ We represent attributes associated with a relationship type using the same symbol as an entity type.
- ❑ However, to distinguish between a relationship with attribute(s) and an entity type, the rectangle representing the attribute(s) is associated with the relationship using a dashed line.



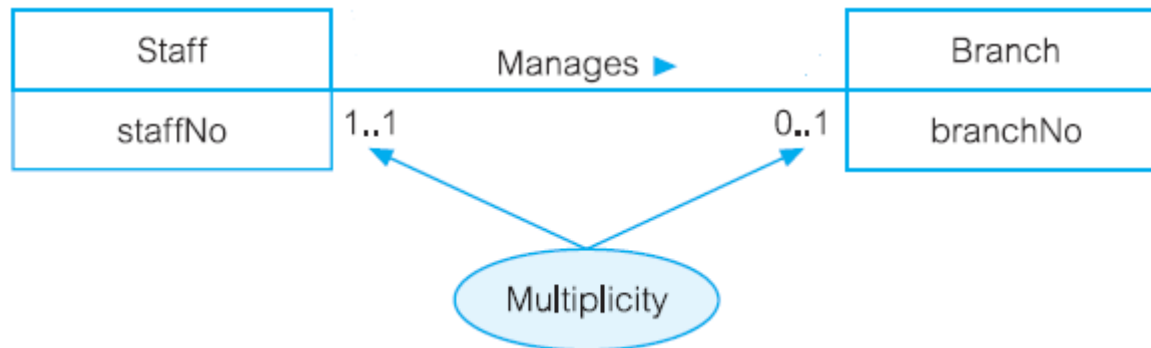
Structural Constraints

- We now examine the constraints that may be placed on entity types that participate in a relationship.

Examples:

- A property for rent must have an owner.
- Each branch must have staff.
- A staff may manage a branch.

- **Multiplicity:** The number (or range) of possible occurrences of an entity type that relate to a single occurrence of an associated entity type through a particular relationship.



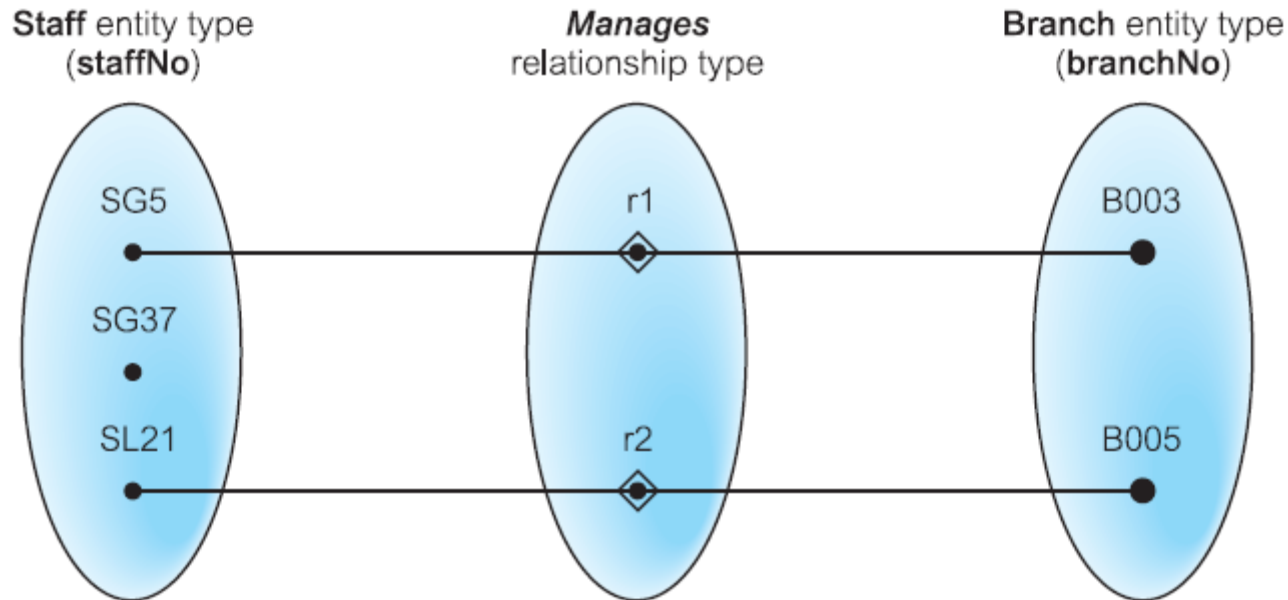
Structural Constraints

- ❑ Binary relationships are generally referred to as being:
 - one-to-one (1:1),
 - one-to-many (1:*),
 - or many-to-many (*:*)).

- ❑ We examine these three types of relationships using the following integrity constraints:
 - a member of staff ***manages*** a branch (1:1);
 - a member of staff ***oversees*** properties for rent (1:*);
 - newspapers ***advertise*** properties for rent (*:*)).

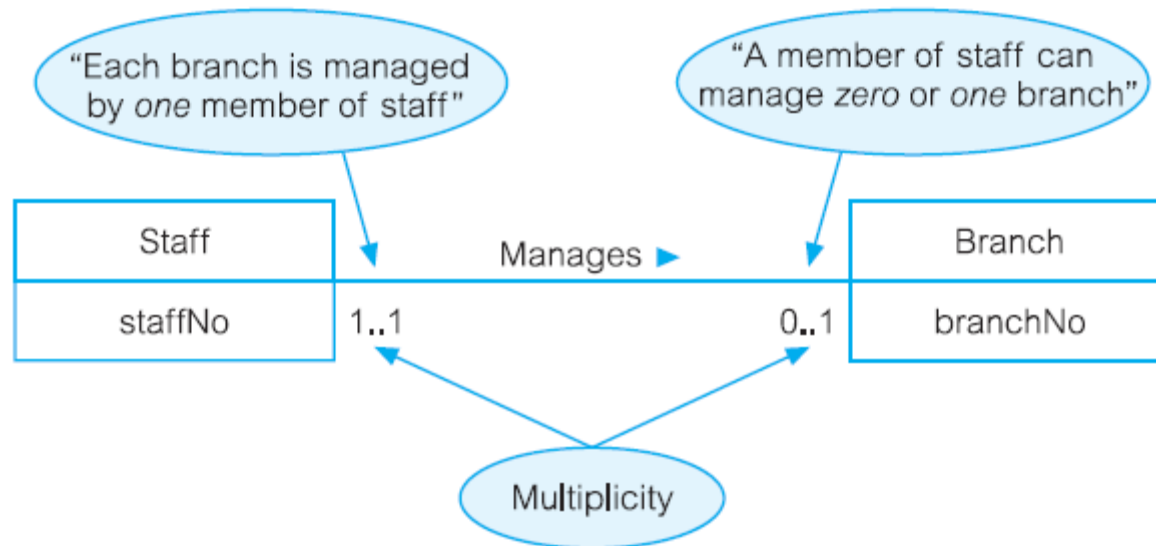
One-to-One (1:1) Relationships

- Consider the relationship *Manages*, which relates the **Staff** and **Branch** entity types.



- A member of staff can manage *zero or one* branch and each branch is managed by *one* member of staff.

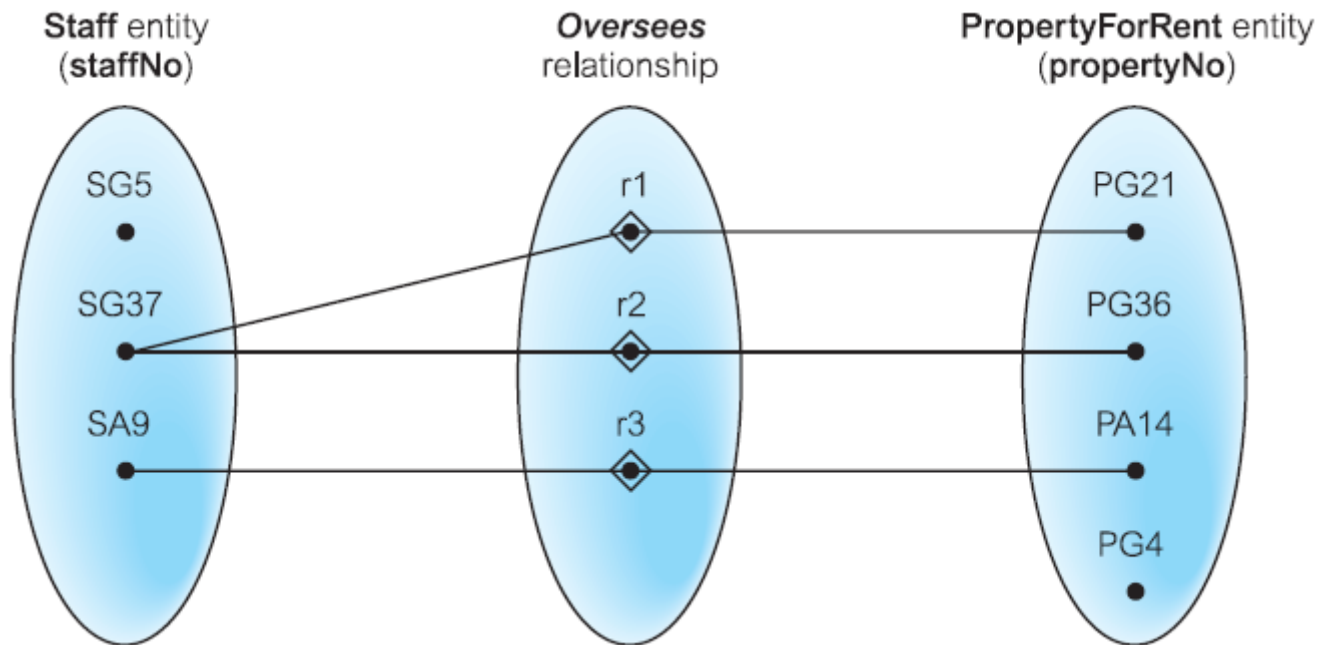
Representation of (1:1) Relationships



- ❑ As there is a maximum of one branch for each member of staff involved in this relationship and a maximum of one member of staff for each branch, we refer to this type of relationship as *one-to-one*, which we abbreviate as (1:1).

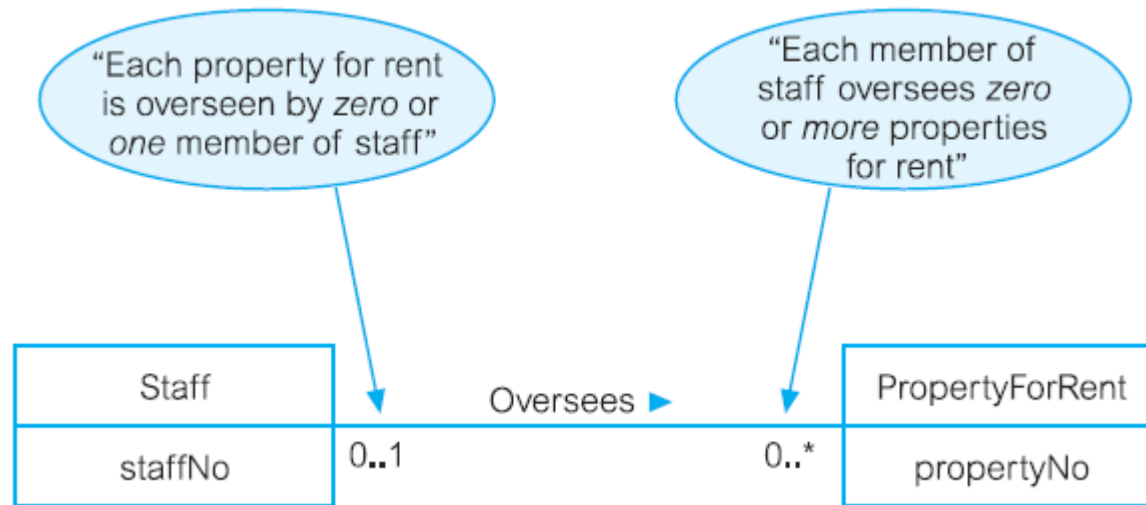
One-to-Many (1:*) Relationships

- Consider the relationship **Oversees**, which relates the **Staff** and **PropertyForRent** entity types.



- A member of staff can oversee *zero or more* properties for rent and a property for rent is overseen by *zero or one* member of staff.

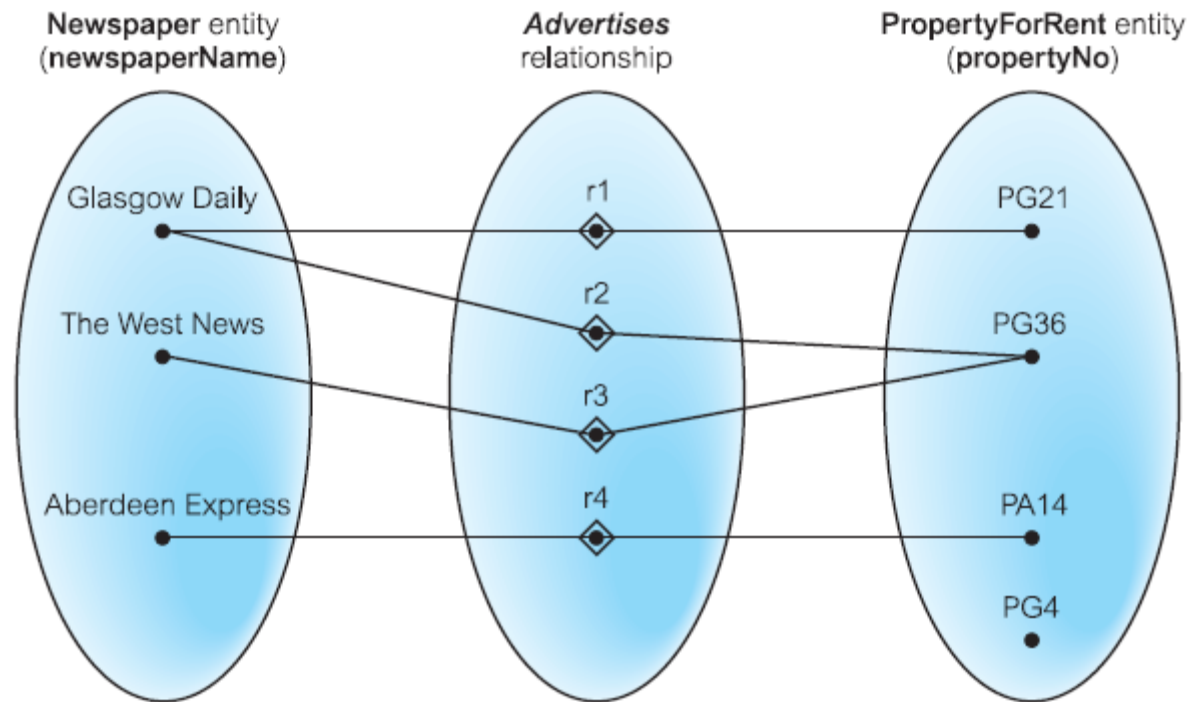
Representation of (1:*) Relationships



- ❑ For members of staff there are *many* properties for rent, and for properties there is a maximum of *one* member of staff. We refer to this type of relationship as *one-to-many*, which we abbreviate as (1:*).
- ❑ If we know the actual minimum and maximum values for the multiplicity, we can display these instead.
For example, we can replace the "0..*" with "0..100".

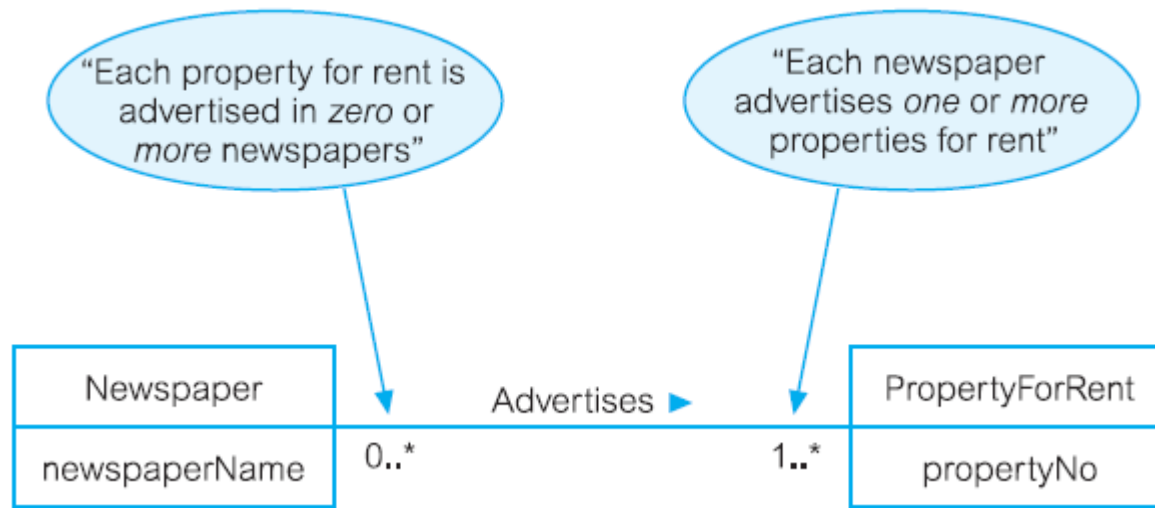
Many-to-Many (*:*) Relationships

- Consider the relationship *Advertises*, which relates the **Newspaper** and **PropertyForRent** entity types.



- A newspaper advertises *one or more* properties for rent and *one* property for rent is advertised in *zero or more* newspapers.

Representation of (*:*) Relationships

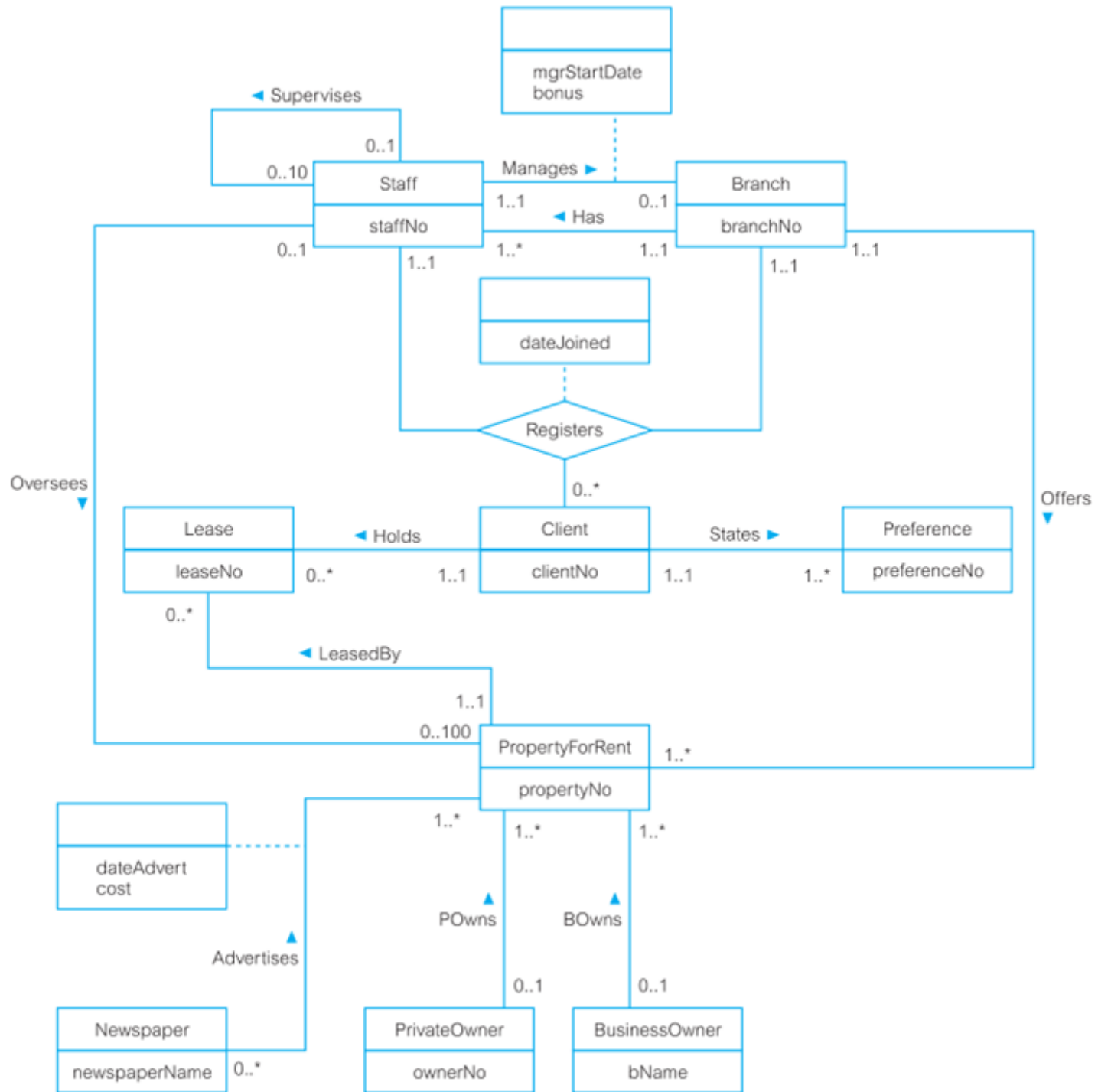


- ❑ For newspapers there are *many* properties for rent, and for each property for rent there are *many* newspapers. We refer to this type of relationship as *many-to-many*, which we abbreviate as $(*:*)$.

Structural Constraints

- A summary of the possible ways that multiplicity constraints can be represented along with a description of the meaning:

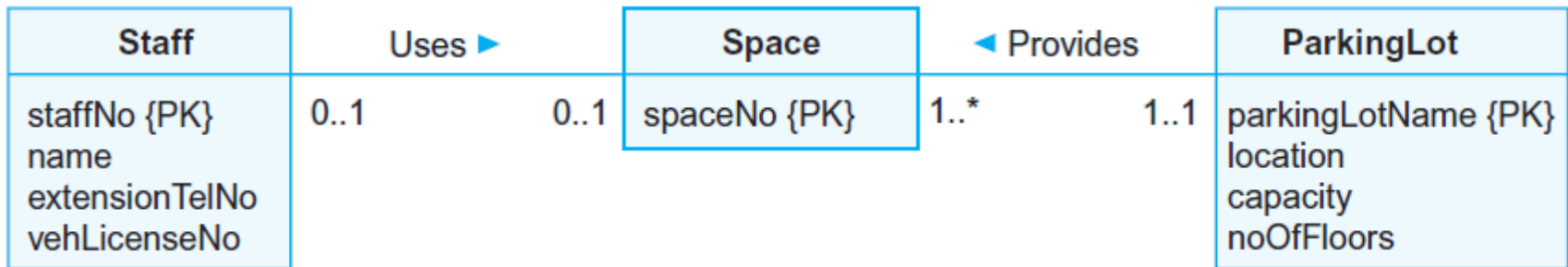
MULTIPLICITY CONSTRAINTS	MEANING
0..1	Zero or one entity occurrence
1..1	Exactly one entity occurrence
0..*	Zero or many entity occurrences
1..*	One or many entity occurrences
5..10	Minimum of 5 up to a maximum of 10 entity occurrences
0, 3, 6–8	Zero or three or six, seven, or eight entity occurrences



Exercise

□ Create an **ER** model for the following descriptions:

- A large organization has several parking lots, which are used by staff.
- Each member of staff has a unique number, name, telephone extension number, and vehicle license number.
- Each parking lot has a unique name, location, capacity, and number of floors.
- Each parking lot has parking spaces, which are uniquely identified using a space number.
- Members of staff can request the sole use of a single parking space.



❑ Create an *ER* model for the following descriptions:

- Departments have employees; and some employees work on projects.
- Each department has a name, a location, and a unique deptNo.
- Each employee has last name, first name, deptNo, and a unique employeeNo.
- A project has a unique projectNo, a name, a budget, and a manager.
- An employee may work on many projects. The enterprise stores the date where the employee entered a project and his/her job in that project.

